

# VIT BHOPAL UNIVERSITY

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

PROJECT REPORT

## STUDENT INFORMATION SYSTEM

Course: Programming in Java

### STUDENT & COURSE INFORMATION

|                 |                  |
|-----------------|------------------|
| Submitted By    | Yuwan Mishra     |
| Register Number | 25BAI10424       |
| Program/Branch  | CSE (AI & ML)    |
| Semester        | Fall Semester    |
| Faculty         | Rizwan ur Rahman |
| Academic Year   | 2026-27          |

## 1. INTRODUCTION

---

The Student Information System is a Java-based terminal application developed to manage student records in a simple, structured, and reliable manner. The application provides a menu-driven console interface through which the user can add, display, search, update, and delete student information seamlessly.

The implementation uses Core Java concepts including classes and objects, encapsulation, ArrayList, methods, loops, conditional statements, exception handling, file handling, and object serialization. Student records are stored locally in `students.dat` so that saved information can be restored when the application is started again.

## 2. PROBLEM STATEMENT

---

Manual management of student records can become difficult as the number of records increases. It may result in duplicate IDs, incorrect GPA values, difficulty locating a student, and loss of information. The proposed system provides a terminal-based solution for maintaining student ID, name, department, and GPA with validation and persistent local storage.

## 3. FUNCTIONAL REQUIREMENTS

---

| ID    | Requirement Description                             |
|-------|-----------------------------------------------------|
| FR-01 | Add a new student record.                           |
| FR-02 | Prevent duplicate student IDs.                      |
| FR-03 | Display all registered students.                    |
| FR-04 | Search a student using ID.                          |
| FR-05 | Update name, department, and optionally GPA.        |
| FR-06 | Delete a student after confirmation.                |
| FR-07 | Accept GPA only from 0.0 to 4.0.                    |
| FR-08 | Handle invalid integer and decimal input.           |
| FR-09 | Load saved records from <code>students.dat</code> . |
| FR-10 | Save records using Java serialization when exiting. |

## 4. NON-FUNCTIONAL REQUIREMENTS

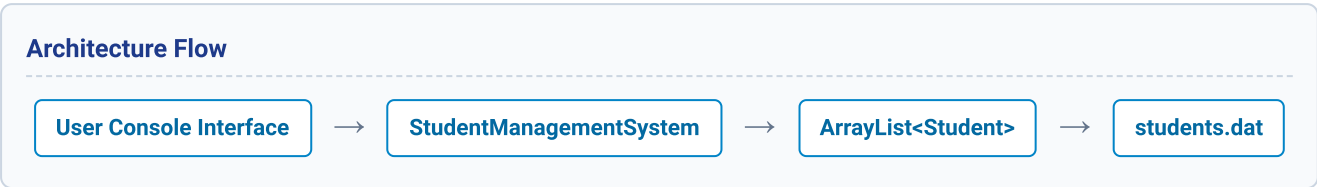
---

- **Usability:** Clear menu-driven terminal interaction with structured prompt messages.
- **Reliability:** Invalid input is gracefully handled using robust validation and exception handling.
- **Maintainability:** Operations are modularized into dedicated, single-responsibility methods.
- **Performance:** Instantaneous response time for standard student datasets.
- **Portability:** Runs seamlessly on any operating system with a compatible Java JDK.
- **Persistence:** Records remain intact across application restarts via binary serialization.

## 5. SYSTEM ARCHITECTURE

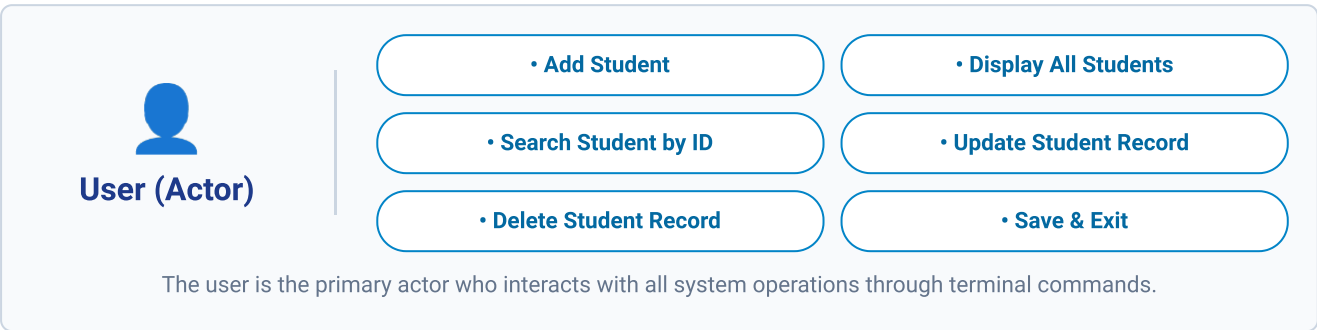
The application follows a layered console-based architecture consisting of user interaction, application logic, in-memory data structures, and persistent file storage.

| Layer         | Component & Responsibility                                                            |
|---------------|---------------------------------------------------------------------------------------|
| USER LAYER    | Terminal / Console Interface for receiving user inputs and displaying text tables.    |
| LOGIC LAYER   | StudentManagementSystem: Handles Menu, CRUD operations, Search, and Input Validation. |
| DATA LAYER    | ArrayList<Student>: Maintains in-memory student records during execution.             |
| STORAGE LAYER | students.dat: Binary persistent file managed via Java Object Serialization.           |

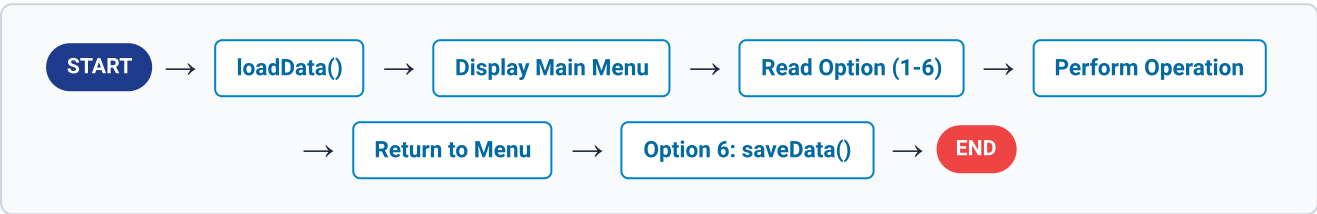


## 6. DESIGN DIAGRAMS

### 6.1 Use Case Diagram



### 6.2 Workflow Diagram



### 6.3 Sequence Diagram - Add Student

| User                 | System (SMS)             | Student Class    | File System           |
|----------------------|--------------------------|------------------|-----------------------|
| Select 'Add Student' | Request ID/Name/Dept/GPA | —                | —                     |
| Provide Inputs       | findStudentById() check  | —                | —                     |
| —                    | Instantiate Object       | new Student(...) | —                     |
| —                    | students.add(student)    | Record in-memory | —                     |
| Select 'Save & Exit' | saveData() execution     | —                | Write to students.dat |

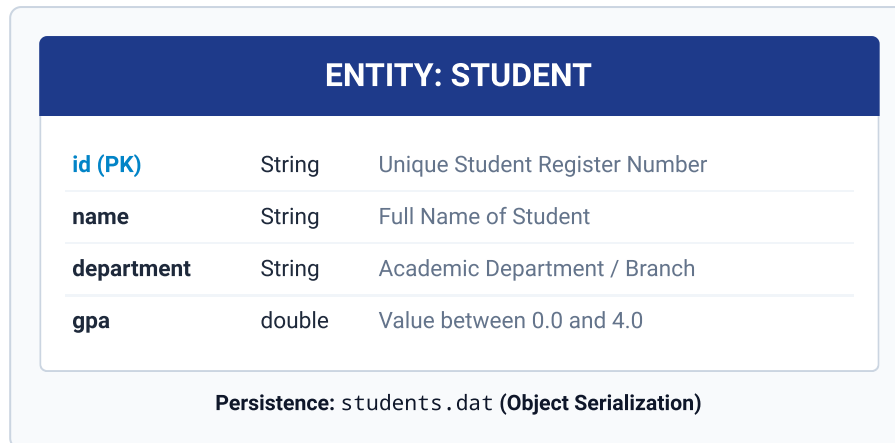
### 6.4 Class / Component Diagram

| Student                                                                                                                                                                                                                                                               | StudentManagementSystem                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>- id : String - name : String - department : String - gpa : double  + Student(...) + getId() : String + getName() : String + getDepartment() : String + getGpa() : double + setName(String) + setDepartment(String) + setGpa(double) + toString() : String</pre> | <pre>- FILE_NAME : String = "students.dat" - students : List&lt;Student&gt; - scanner : Scanner  + main(String[]) : void + printMenu() : void + addStudent() : void + viewAllStudents() : void + searchStudent() : void + updateStudent() : void + deleteStudent() : void + loadData() : void + saveData() : void</pre> |

**Relationship:** StudentManagementSystem maintains a 1-to-many composition collection of Student instances managed via ArrayList<Student>.

## 6.5 ER Diagram / Data Model

The project does not use an external relational database management system (RDBMS). Instead, it stores serialized Student objects locally. The logical data model is represented by the single entity model below:



## DATA MODEL SPECIFICATIONS

Each Student entity represents an individual academic record. Field attributes are strictly encapsulated using Java's private modifier access control. Serialization is enabled by implementing the `java.io.Serializable` interface, providing a unique runtime signature (`serialVersionUID`).

- **Primary Identifier:** `id` serves as the primary key. Case-insensitive matching is enforced to guarantee uniqueness across all transactions.
- **Validation Rules:** The `gpa` attribute is constrained within the closed range `[0.0, 4.0]`. Any input outside this bound is rejected during object instantiation.

## 7. DESIGN DECISIONS & RATIONALE

| Decision                | Rationale                                                                                                  |
|-------------------------|------------------------------------------------------------------------------------------------------------|
| Terminal Interface      | Keeps the project focused on Core Java fundamentals and algorithmic logic without GUI overhead.            |
| ArrayList<Student>      | Provides efficient dynamic memory management and indexed search capabilities for record handling.          |
| Serializable Interface  | Allows direct, seamless object graph writing to disk without requiring manual text parsing.                |
| students.dat File       | Provides lightweight, zero-dependency local data persistence without requiring external database setup.    |
| Case-Insensitive Search | Enhances user experience by allowing register number lookup regardless of uppercase/lowercase input.       |
| GPA Validation Loop     | Prevents corrupt data entry by ensuring GPA strictly adheres to the standard 0.0 to 4.0 grading scale.     |
| Modular Methods         | Ensures clean Separation of Concerns (SoC), improving overall readability, debugging, and maintainability. |

## 8. IMPLEMENTATION DETAILS

**Student class:** Implements `Serializable` and encapsulates four private attributes: `id`, `name`, `department`, and `gpa`. A parameterized constructor initializes new instances. Public getter and setter methods provide controlled access, while an overridden `toString()` method formats output cleanly for terminal presentation.

**StudentManagementSystem class:** Serves as the driver class, maintaining the central `ArrayList<Student>` collection and a `Scanner` object. The `main()` method invokes `loadData()` on start, displays the interactive menu, routes operational choices, and executes `saveData()` upon exiting.

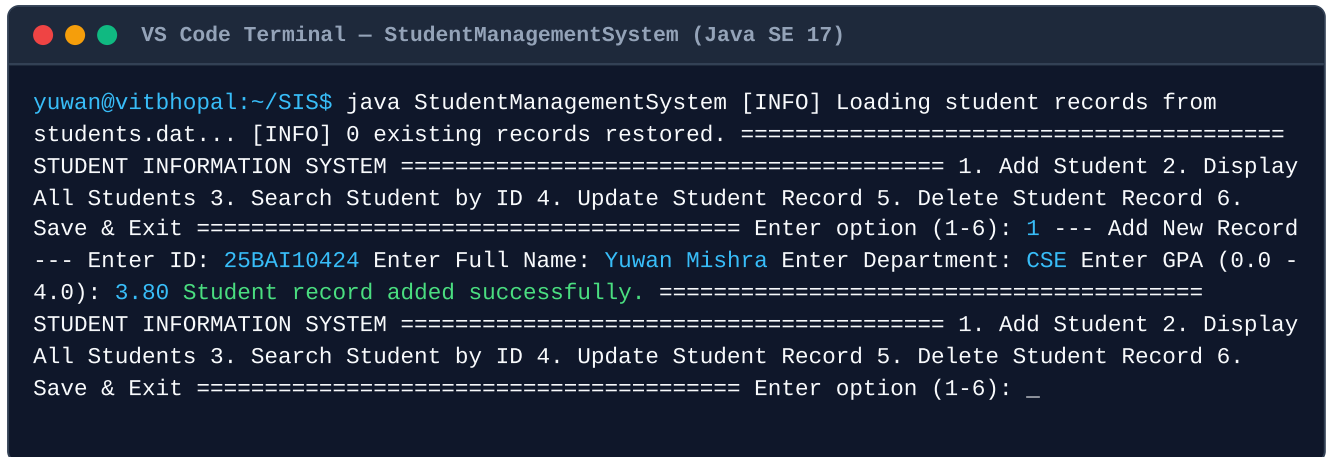
**CRUD Operations:** `addStudent()` checks for duplicate IDs before insertion. `viewAllStudents()` renders records in tabular format. `searchStudent()` performs linear matching against stored IDs. `updateStudent()` modifies fields selectively, and `deleteStudent()` removes entries upon explicit confirmation.

**Input Validation & Exception Handling:** Custom loop control functions (`readIntInput()`, `readDoubleInput()`) catch `InputMismatchException` to handle invalid character inputs gracefully without crashing the JVM.

## 9. SCREENSHOTS / RESULTS

---

The following terminal output captures were executed within the VS Code development environment. They demonstrate execution flow, record creation, updating, deletion, and persistent storage verification.



```
VS Code Terminal – StudentManagementSystem (Java SE 17)

yuwan@vitbhopal:~/SIS$ java StudentManagementSystem [INFO] Loading student records from
students.dat... [INFO] 0 existing records restored. =====
STUDENT INFORMATION SYSTEM ===== 1. Add Student 2. Display
All Students 3. Search Student by ID 4. Update Student Record 5. Delete Student Record 6.
Save & Exit ===== Enter option (1-6): 1 --- Add New Record
--- Enter ID: 25BAI10424 Enter Full Name: Yuwan Mishra Enter Department: CSE Enter GPA (0.0 -
4.0): 3.80 Student record added successfully. =====
STUDENT INFORMATION SYSTEM ===== 1. Add Student 2. Display
All Students 3. Search Student by ID 4. Update Student Record 5. Delete Student Record 6.
Save & Exit ===== Enter option (1-6): _
```

Figure 1. Adding student record for Yuwan Mishra (Register No: 25BAI10424).

```
VS Code Terminal – Registered Student List View

Enter option (1-6): 1 --- Add New Record --- Enter ID: 25BAI10012 Enter Full Name:
Aarav Sharma Enter Department: ECE Enter GPA (0.0 - 4.0): 3.65 Student record added
successfully. ===== STUDENT INFORMATION SYSTEM
===== 1. Add Student 2. Display All Students 3. Search
Student by ID 4. Update Student Record 5. Delete Student Record 6. Save & Exit
===== Enter option (1-6): 2 -----
----- | ID | Name | Department | GPA | -----
----- | 25BAI10424 | Yuwan Mishra | CSE | 3.80 | |
25BAI10012 | Aarav Sharma | ECE | 3.65 | -----
----- Total Students Registered: 2 ===== STUDENT
INFORMATION SYSTEM ===== 1. Add Student 2. Display All
Students 3. Search Student by ID 4. Update Student Record 5. Delete Student Record 6. Save &
Exit ===== Enter option (1-6): _
```

Figure 2. Displaying registered students including Yuwan Mishra (25BAI10424) in tabular output format.

```
VS Code Terminal – Record Update and Deletion Routine

Enter option (1-6): 4 Enter Student ID to update: 25BAI10424 Current Record: ID:
25BAI10424 | Name: Yuwan Mishra | Dept: CSE | GPA: 3.80 New Name (press Enter to keep 'Yuwan
Mishra'): Yuwan Mishra New Department (press Enter to keep 'CSE'): CSE (AI & ML) Update GPA?
(y/n): y Enter new GPA (0.0 - 4.0): 3.92 Record updated successfully.
===== STUDENT INFORMATION SYSTEM
===== 1. Add Student 2. Display All Students 3. Search
Student by ID 4. Update Student Record 5. Delete Student Record 6. Save & Exit
===== Enter option (1-6): 5 Enter Student ID to delete:
25BAI10012 Confirm deletion of Aarav Sharma (y/n): y Record deleted successfully.
===== Enter option (1-6): 6 [INFO] Saving records to
students.dat... Data saved successfully. Exiting system. Goodbye!
```

Figure 3. Updating and deleting student records in the terminal console.

## 10. TESTING APPROACH

---

Testing was conducted using manual test cases covering functional boundary limits, exception handling, menu option validation, and file persistence verification.

| TC    | Test Input / Action                      | Expected Result                                  | Status |
|-------|------------------------------------------|--------------------------------------------------|--------|
| TC-01 | Valid student (25BAI10424, Yuwan Mishra) | Record added successfully to memory.             | PASS   |
| TC-02 | Duplicate ID (25BAI10424)                | Duplicate ID rejected with warning message.      | PASS   |
| TC-03 | GPA out of bounds (-1.0 or 4.5)          | Value rejected; prompt requests input again.     | PASS   |
| TC-04 | Display all students                     | All stored records rendered in formatted table.  | PASS   |
| TC-05 | Search existing ID (25BAI10424)          | Matching student details displayed accurately.   | PASS   |
| TC-06 | Search non-existent ID                   | "No student found" message presented.            | PASS   |
| TC-07 | Update existing record fields            | Selected fields updated; unchanged preserved.    | PASS   |
| TC-08 | Delete record with 'y' confirm           | Record removed from array list.                  | PASS   |
| TC-09 | Delete record with 'n' confirm           | Deletion cancelled; record remains intact.       | PASS   |
| TC-10 | Restart system after Save & Exit         | Saved records automatically reloaded from disk.  | PASS   |
| TC-11 | Non-numeric menu input ("abc")           | Input error caught gracefully; menu redisplayed. | PASS   |

## 11. CHALLENGES FACED

---

- Handling invalid numeric inputs without causing infinite scanner buffer loops during input exception catching.
- Enforcing unique student register numbers across case-insensitive operational checks.
- Managing dynamic updating and safe element removal inside dynamic `ArrayList` collections.
- Configuring Java Object Serialization and handling potential `ClassNotFoundException` on deserialization.

## 12. LEARNINGS & KEY TAKEAWAYS

---

- Applied Core Object-Oriented Programming (OOP) principles in an end-to-end practical project.
- Mastered data encapsulation using private class members and public accessor methods.
- Gained hands-on proficiency with the Java Collections Framework (specifically `ArrayList`).
- Implemented robust exception handling and validated user inputs safely.
- Understood object stream persistence using `ObjectInputStream` and `ObjectOutputStream`.

## 13. FUTURE ENHANCEMENTS

---

- Incorporate role-based access control (RBAC) separating Admin and Student permissions.
- Integrate a relational database like MySQL or PostgreSQL via JDBC for scalable storage.
- Develop a graphical user interface (GUI) using JavaFX or Swing, or transition to a Spring Boot Web API.
- Export student transcript summaries directly to formatted PDF or CSV formats.

## 14. REFERENCES

---

- Oracle Java Documentation — Standard Library API Specifications & Guides.
- Java SE Documentation — Collections Framework, I/O Streams, and Object Serialization.
- Programming in Java Course Material — School of Computing Science and Engineering, VIT Bhopal University.

## 15. CONCLUSION

---

The Student Information System developed by **Yuwan Mishra** (Reg No: **25BAI10424**) successfully demonstrates how Core Java principles can be applied to solve real-world record management challenges. The application fulfills all functional requirements including full CRUD operations, input sanitization, case-insensitive ID lookups, and persistent file storage. The codebase provides an extensible base for future enterprise features such as database integration and modern GUI interfaces.